View Naming	
URL	http://../store/list
File	app/views/store/list.html.erb (or .builder)
Helper	module StoreHelper
File	app/helpers/store_helper.rb

There's one extra twist. In normal Ruby code you have to use the `require` keyword to include Ruby source files before you reference the classes and modules in those files. Since Rails knows the relationship between filenames and class names, `require` isn't normally necessary in a Rails application. The first time you reference a class or module that isn't known, Rails uses the naming conventions to convert the class name to a filename and tries to load that file behind the scenes. The net effect is that you can typically reference (say) the name of a model class, and that model will be automatically loaded into your application.

Grouping Controllers into Modules

So far, all our controllers have lived in the `app/controllers` directory. It's sometimes convenient to add more structure to this arrangement. For example, our store might end up with a number of controllers performing related but disjoint administration functions. Rather than pollute the top-level namespace, we might choose to group them into a single `admin` namespace.



David says:

Why Plurals for Tables?

Because it sounds good in conversation. Really. "Select a Product from products." And "Order has_many :line_items."

The intent is to bridge programming and conversation by creating a domain language that can be shared by both. Having such a language means cutting down on the mental translation that otherwise confuses the discussion of a *product description* with the client when it's really implemented as *merchandise body*. These communications gaps are bound to lead to errors.

Rails sweetens the deal by giving you most of the configuration for free if you follow the standard